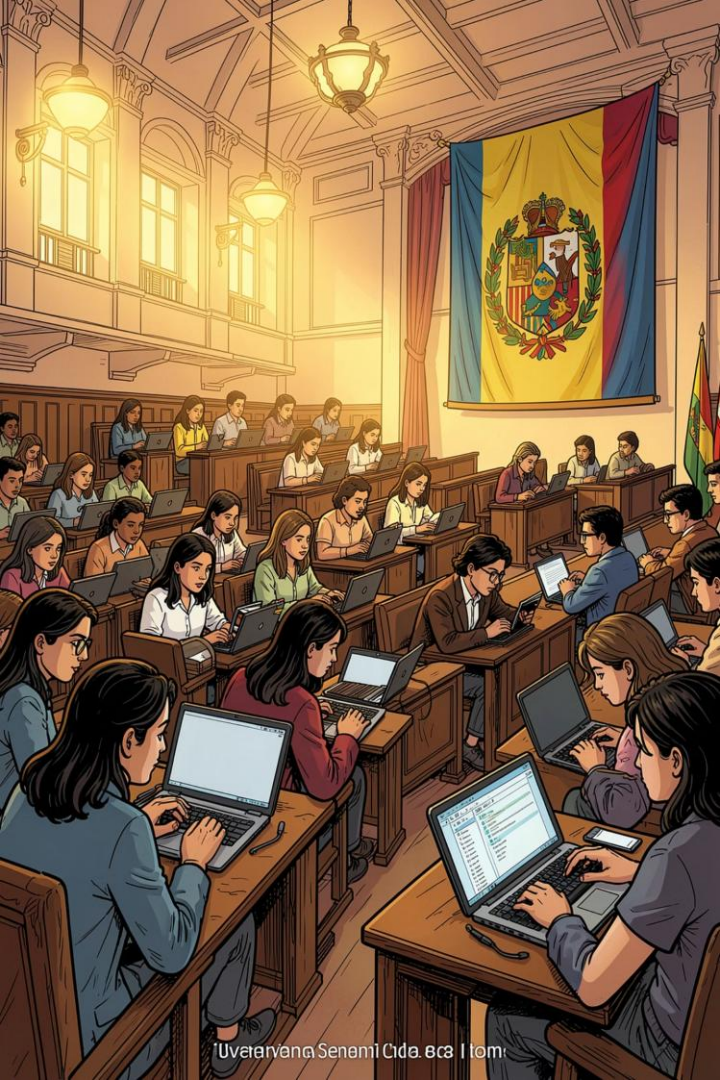




PROYECTO INTEGRADOR

El impacto de las nuevas tecnologías en la sociedad: desarrollo y proyección de soluciones informáticas

Juego: Piedra, Papel o Tijera — proyecto académico que integra conceptos de lógica y programación aplicados a una solución práctica en Python.

Universidad

Universidad
Internacional del
Ecuador — Facultad
de Ingeniería y
Ciencias Aplicadas

Programa

Escuela de
Ciberseguridad —
Lógica de
Programación I

Autor

Noe Paúl Llanganate
Zavala

Fecha

28 Junio 2026

Descripción del problema

Las nuevas tecnologías transforman enseñanza y práctica profesional. En el contexto universitario es imprescindible que el aprendizaje de programación se complemente con proyectos reales que fomenten pensamiento lógico, diseño de algoritmos y buenas prácticas de desarrollo.



Problema

Necesidad de aplicar contenidos de la asignatura en un proyecto funcional que demuestre diseño lógico, control de flujo y manejo de datos.



Objetivo

Desarrollar una aplicación en Python que integre estructuras lógicas, repetitivas, funciones y registro de resultados.

Objetivos

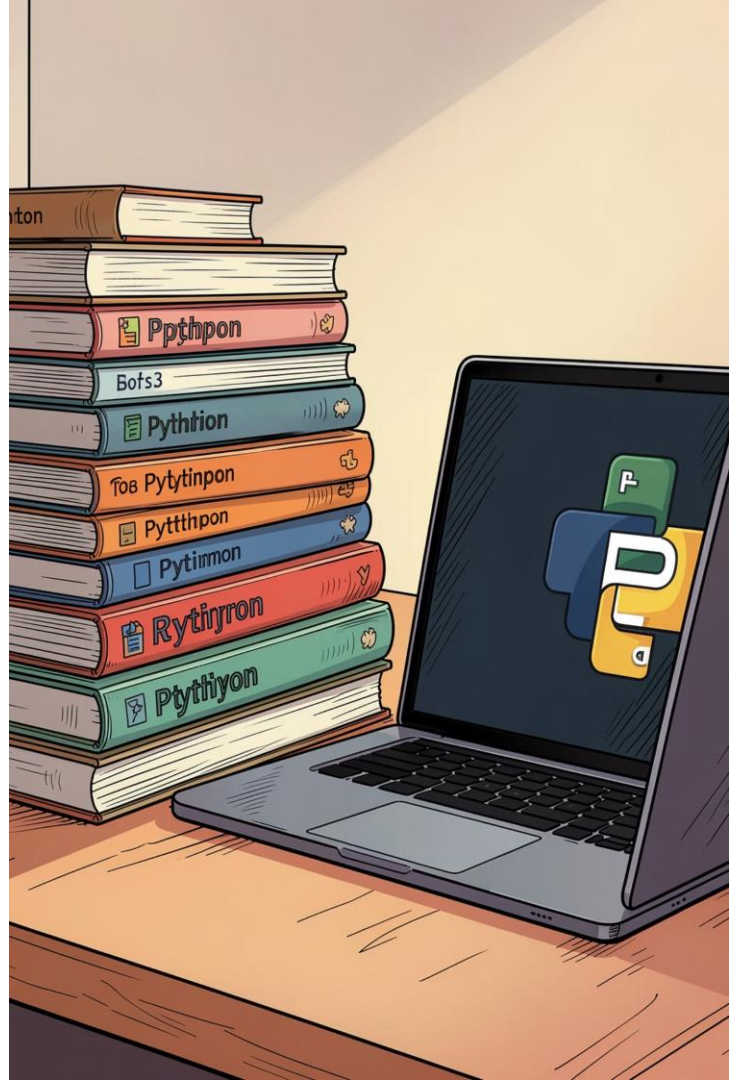
Objetivo General

Desarrollar un software funcional utilizando Python que permita jugar Piedra, Papel o Tijera, registrar resultados y manejar interacción usuario-CPU.

Objetivos específicos

- Aplicar estructuras lógicas (condicionales) para la toma de decisiones del juego.
- Utilizar estructuras repetitivas para el flujo de partidas y menús.
- Organizar el código mediante funciones claras y reutilizables.
- Implementar control de versiones con GitHub para seguimiento y colaboración.

Estos objetivos permiten evaluar competencias técnicas y metodológicas: análisis del problema, diseño modular, y uso de herramientas colaborativas.



Herramientas utilizadas

Para el desarrollo se seleccionaron herramientas estándar en ingeniería de software que facilitan programación, control de versiones y despliegue en entornos de práctica.

Herramienta	Uso
Python	Desarrollo del juego y lógica del programa
Visual Studio Code	Edición de código, depuración y extensiones (lint, formatter)
Git	Control de versiones local y gestión de ramas
GitHub	Repositorio remoto, seguimiento de issues y documentación
Windows 11	Plataforma de ejecución y pruebas

Recomendación: incluir README con instrucciones y ejemplos de ejecución para facilitar la evaluación y replicación.

OBER UHEBU..

MENU



GAME LOGIC



CPU RANDOM GENERATOR ?1080

HISTORY



Arquitectura del sistema

Arquitectura modular sencilla diseñada para claridad: la interfaz de menú dirige al juego; el motor de juego genera la jugada de la CPU, compara resultados, actualiza historial y marcador.

Usuario

Menú Principal

Juego

Generador CPU

Modularidad facilita pruebas unitarias (funciones independientes) y extensibilidad (por ejemplo, añadir modos de juego o persistencia).

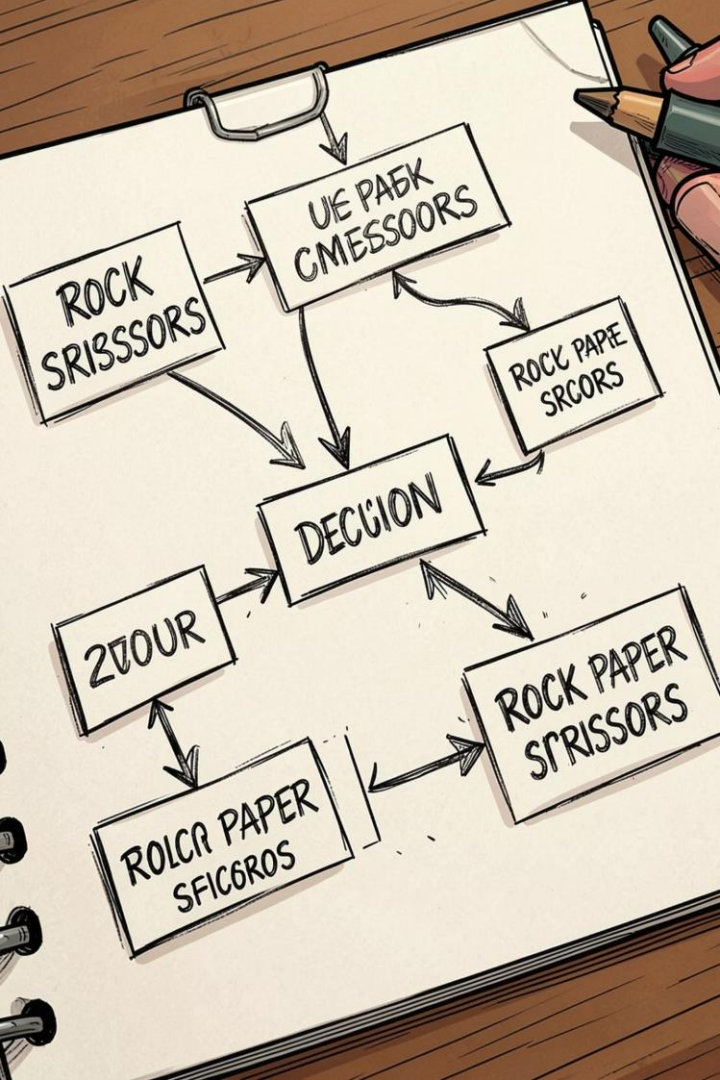


Diagrama de flujo

El flujo muestra inicio → menú → selección del jugador → generación aleatoria de la CPU → comparación de jugadas → actualización de historial/marcador → opción de repetir o salir.



Explicación: cada decisión usa condicionales anidados (if/elif/else) y bucles while para mantener la ejecución hasta que el usuario decida salir.

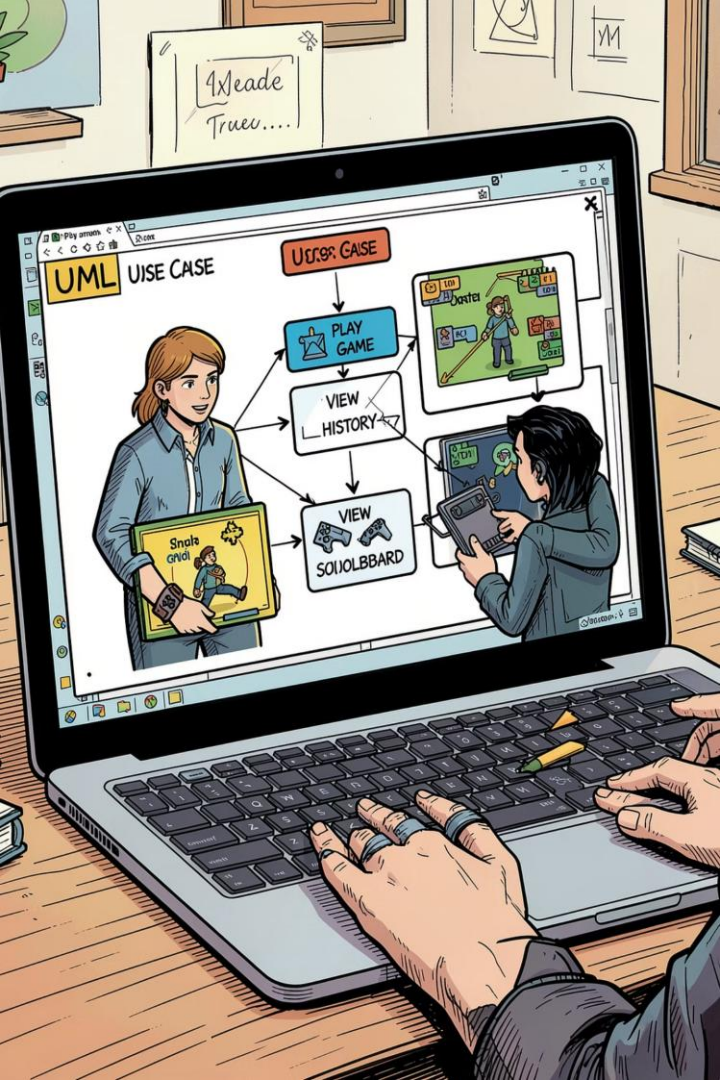


Diagrama de casos de uso

Representación UML que identifica al actor principal (Usuario) y sus interacciones: jugar partida, consultar historial y ver marcador. El sistema ejecuta la lógica y registra resultados.

Jugar partida

Iniciar y controlar una partida en el sistema

Consultar historial

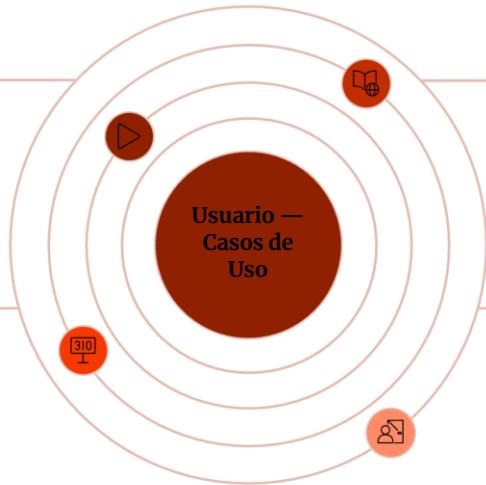
Ver partidas previas y resultados del usuario

Visualizar marcador

Mostrar puntuación actual y ranking

Salir

Cerrar sesión o finalizar la aplicación



Importancia: clarifica requisitos funcionales y límites del sistema, útil para pruebas y planificación de entregables.

Funcionalidades

El programa incluye las funcionalidades esenciales para una experiencia completa e interactiva, con registro de partidas y marcador visible durante la ejecución.



Menú principal

Navegación clara entre opciones: jugar, historial, marcador y salir.



Piedra

Opción de jugada para el usuario.



Papel

Opción de jugada para el usuario.



Tijera

Opción de jugada para el usuario.



Historial

Registro de partidas jugadas con jugadas y resultado por ronda.



Marcador

Conteo acumulado de victorias, derrotas y empates.



Salir

Finalizar sesión y mostrar resumen de la partida.



CPU Aleatoria

Generador aleatorio que simula la jugada de la computadora usando random.


```
Python 3.10.0 (tags/v3.10.0:9006666, Dec 12, 2022)
Python 3.10.0 (tags/v3.10.0:9006666, Dec 12, 2022)

def pypaint():
    if else {
        if elif("0")
        pxurt:ontuicsil(foetn**lp,")
    }

    if elif' else {
        ll siier liud( {f'unnut ""(ralu)":
    }

    while
    while( )
    }

    oblf gfrelxss )
    ipce
    nnile( "0"):
        lxle (=fan)av )):
    }

    import random("9=")):
    }

    import radoow(raunt,.0")){
    }

    impntt oqp):
    }

    ol:att( )
    poonr"():
    }

    }
```

Código fuente

El código se organiza en funciones (por ejemplo: mostrar_menu(), jugar_ronda(), obtener_jugada_cpu(), comparar_jugadas(), mostrar_historial()). Emplea estructuras condicionales (if/elif/else), bucles while para el ciclo principal y el módulo random para la CPU.

Estructuras usadas

- Funciones: modularidad y reutilización.
- Condicionales: decisión del ganador.
- Bucle while: mantener la ejecución del menú.
- random.choice(): selección aleatoria de la CPU.

Enlace al archivo

Captura del archivo principal:
[main.py](#)

Recomendación práctica: agregar comentarios y pruebas simples (cases) para documentar comportamiento esperado de cada función.

Ejecución y demostración

Durante la ejecución, el usuario selecciona una opción en el menú, el programa solicita la jugada, la CPU responde aleatoriamente y se compara el resultado. El marcador y el historial se actualizan en cada ronda.

Paso 1 — Menú

Mostrar opciones: jugar, historial, marcador, salir.

Paso 3 — Resultado

Comparar jugadas con if/elif/else, actualizar marcador e historial, mostrar mensaje: victoria/derrota/empate.

Paso 2 — Jugar

Leer la jugada del usuario y generar la jugada de la CPU con random.

Paso 4 — Repetir o salir

Volver al menú principal hasta que el usuario seleccione salir; al cerrar, mostrar resumen final.

Sugerencia final: versionar en GitHub con commits claros y un README que incluya instrucciones de instalación (por ejemplo: crear entorno virtual, instalar dependencias si aplica, comando de ejecución).